

Embedded C met Arduino

Inhoud en leerlijn

Dit traject bouwt verder op klassieke Arduino-programma's, maar verlaat bewust de gewoonte om alles in één grote loop() met delay() te stoppen. Je leert de basis van firmware schrijven als een verzameling kleine, voorspelbare modules die samenwerken via tijd, events en toestanden.

Hoofdstuk	Kernidee	Resultaat
1. Van blocking naar non-blocking	delay(), while-wachten en gemiste events	De cursist ervaart waarom blocking code beperkt is.
2. Tijd beheeren met millis()	Periodieke acties zonder delay()	Meerdere taken lijken tegelijk te lopen.
3. Cooperative scheduler	Takenlijst, perioden, function pointers	Een kleine taakplanner ontstaat.
4. Timer tick en software timers	Hardwaretimer als systeemtijd	Meer realistische RTOS-denkwijze.
5. Events en event queue	Producer-consumer, buffering, asynchroon gedrag	Loskoppeling tussen detecteren en verwerken.
6. FSM basis	States, events, transitions	Gedrag wordt expliciet gemodelleerd.
7. Event-driven FSM	FSM reageert op queue-events	Geen vlaggenchaos, maar gecontroleerde transities.
8. Meerdere FSM-modules	UI, motor, proces, alarm als aparte FSM	Modulaire firmware.
9. Integratiearchitectuur	Scheduler + events + FSM + drivers	Mini embedded framework.
10. Grote oefening: wasmachine	Volledig systeem met timing, sensorfouten en bediening	Eindproject dat alles samenbrengt.

Didactisch uitgangspunt

De volgorde is bewust gekozen. Eerst ontstaat het probleem: zodra meerdere dingen tegelijk moeten gebeuren, faalt blocking code. Daarna komt de scheduler als oplossing voor timing en taakverdeling. Vervolgens ontstaat nieuwe complexiteit: veel flags, veel if-structuren, veel verborgen toestand. Dan wordt FSM de natuurlijke oplossing voor gedrag.

De rode draad is dus niet “we leren twee aparte onderwerpen”, maar: we bouwen stap voor stap een embedded architectuur.

Tinkercad-opstelling

Alle oefeningen kunnen in Tinkercad met een Arduino Uno worden uitgevoerd. Voor de wasmachine-oefening volstaan eenvoudige componenten: drukknoppen, LEDs, potentiometer, DC-motor of motor-simulatie via LED/PWM, buzzer en eventueel een LCD indien gewenst.

Onderdeel	Simulatie in Tinkercad	Didactische rol
Startknop	Pushbutton met pull-up	Gebruikerscommando
Deurcontact	Pushbutton of schakelaar	Veiligheidsvoorwaarde
Waterniveau	Potentiometer of schakelaar	Sensorwaarde / fault
Motor	DC-motor of LED met PWM	Actuator
Pomp/klep	LED	Actuatorstatus
Buzzer	Piezo	Feedback en alarm
Seriële monitor	Serial	Debug-output en command shell

1. Van blocking naar non-blocking programmeren

Een klassiek Arduino-programma werkt vaak zolang er maar één activiteit is. Zodra een programma tegelijk op knoppen moet reageren, een motor moet aansturen, seriële commando's moet verwerken en LEDs moet updaten, wordt delay() een structureel probleem.

Theoretische onderbouw

Blocking code is code die de processor vasthoudt terwijl er eigenlijk niets nuttigs gebeurt. delay(1000) betekent niet dat de Arduino "slim wacht"; het betekent vooral dat de rest van het programma gedurende die periode niet verder kan. Ook while-lussen die wachten op een knop, seriële input of een sensorconditie zijn blocking.

Typische symptomen

- Knopdrukken worden gemist.
- Seriële input wordt te laat gelezen.
- Motorregeling reageert schokkerig.
- LED-patronen verstoren andere functies.
- Het programma wordt moeilijk uitbreidbaar.

Slecht voorbeeld

```
void loop() {  
    digitalWrite(13, HIGH);  
    delay(1000);  
    digitalWrite(13, LOW);  
    delay(1000);  
  
    if (digitalRead(2) == LOW) {  
        Serial.println("Knop ingedrukt");  
    }  
}
```

De knop wordt alleen gelezen op twee korte momenten per cyclus. Een druk tijdens delay() wordt niet verwerkt.

Vrijblijvende oefening

Implementeer bovenstaande code om een LED te laten knipperen met delay() en tegelijk een drukknop te registreren.

Probeer een aantal korte drukken te registreren, wat vanzelf de vaststelling oplevert dat het programma niet responsief is.

2. Tijd beheeren met millis()

De eerste stap naar een scheduler is het loslaten van wachten...delay().

In plaats van te wachten tot een tijd voorbij is, kijkt het programma telkens of een bepaalde tijd al verstreken is.

Basispatroon

```
unsigned long vorigeTijd = 0;  
const unsigned long interval = 500;  
  
void loop() {  
    unsigned long nu = millis();
```

```
if (nu - vorigeTijd >= interval) {  
    vorigeTijd = nu;  
    digitalWrite(13, !digitalRead(13));  
}  
  
// Andere code blijft doorlopen  
leesKnop();  
verwerkSerial();  
}
```

Waarom nu - vorigeTijd?

Het patroon `nu - vorigeTijd >= interval` blijft correct wanneer `millis()` na lange tijd overloopt. Dat is een belangrijk embedded inzicht: vermijd absolute vergelijkingen zoals `nu >= vorigeTijd + interval`.

Oefeningen

1. Maak drie LEDs die onafhankelijk knipperen met drie verschillende perioden.
2. Voeg een drukknop toe die op elk moment de snelste LED aan of uit zet.
3. Stuur elke seconde een statusregel naar Serial zonder de knipperende LEDs te verstoren.
4. Meet hoe responsief het programma blijft bij korte knopdrukken.

3. Cooperative Scheduler / Mini RTOS

Een cooperative scheduler is een eenvoudige taakplanner. Hij beslist niet met dwang wanneer een taak moet stoppen. Elke taak moet zelf kort blijven en snel terugkeren. Daarom heet dit cooperative: de taken werken vrijwillig mee.

Theoretische onderbouw

Een echt RTOS kan taken preemptief onderbreken. Op Arduino Uno is het interessant om eerst een cooperative scheduler te bouwen. Daardoor zie je helder wat taken, perioden, deadlines en jitter betekenen, zonder de complexiteit van context switching.

Begrip	Betekenis
Task	Een kleine functie met één duidelijke verantwoordelijkheid.
Period	Hoe vaak de taak idealiter uitgevoerd wordt.
Deadline	Wanneer de taak uiterlijk klaar moet zijn.
Jitter	Variatie op het ideale uitvoeringsmoment.
Starvation	Een taak komt te weinig of te laat aan bod door andere taken.
Cooperative	Taken blokkeren niet en geven snel controle terug.

Eerste scheduler met vaste functies

```
void loop() {  
    taskLed();  
    taskButtons();  
    taskSerial();  
    taskMotor();  
}
```

Dit is nog geen echte scheduler, maar het is al een architecturale verbetering: verantwoordelijkheden worden gescheiden.

Scheduler met taakstructuur

```
typedef void (*TaskFunction)(void);  
  
typedef struct {  
    TaskFunction function;  
    unsigned long period;  
    unsigned long lastRun;  
} Task;  
  
void taskLed(void);  
void taskButtons(void);  
void taskSerial(void);  
  
Task tasks[] = {  
    { taskLed,    500, 0 },  
    { taskButtons, 10, 0 },  
    { taskSerial, 20, 0 }  
};  
  
const byte taskCount = sizeof(tasks) / sizeof(tasks[0]);
```

```

void schedulerRun(void) {
    unsigned long now = millis();

    for (byte i = 0; i < taskCount; i++) {
        if (now - tasks[i].lastRun >= tasks[i].period) {
            tasks[i].lastRun = now;
            tasks[i].function();
        }
    }
}

void loop() {
    schedulerRun();
}

```

Belangrijke bespreking

- Elke task moet kort zijn.
- Een task mag geen delay() bevatten.
- Een task mag niet eindeloos in een while wachten.
- Taken mogen globale toestand gebruiken, maar dat moet bewust gebeuren.
- Debug-output via Serial kan zelf timing beïnvloeden.

4. Timer tick en software timers

In een volgende stap wordt de systeemtijd niet meer uitsluitend via `millis()` gebruikt, maar bouwen we conceptueel een eigen tick. Dat brengt je dichterbij de manier waarop RTOS-systemen werken.

Timer interrupt als systeemtick

Een hardwaretimer kan periodiek een interrupt veroorzaken. In die ISR verhogen we een teller. De hoofdcode gebruikt die teller als tijdsbasis. De ISR blijft extreem kort.

```
volatile unsigned long systemTicks = 0;
```

```
ISR(TIMER1_COMPA_vect) {  
    systemTicks++;  
}
```

```
unsigned long getTicks(void) {  
    unsigned long copy;  
    noInterrupts();  
    copy = systemTicks;  
    interrupts();  
    return copy;  
}
```

De `noInterrupts()/interrupts()`-sectie is nodig omdat een `unsigned long` op een 8-bit AVR niet atomisch gelezen wordt.

CTC-idee zonder te verdrinken in registers

De timer telt op tot een compare value. Bij match wordt een interrupt gegenereerd en begint de teller opnieuw. Dat levert een regelmatige tick op. Dit kan desgewenst ook via port registers.

Software timers

Een software timer is een klein object dat bijhoudt wanneer een event moet afgaan. FSMs gebruiken software timers vaak voor timeouts: deur 3 seconden open, wassen 10 seconden, spoelen 5 seconden, enzovoort.

```
typedef struct {  
    unsigned long start;  
    unsigned long duration;  
    bool active;  
} SoftTimer;  
  
void timerStart(SoftTimer *t, unsigned long duration) {  
    t->start = getTicks();  
    t->duration = duration;  
    t->active = true;  
}  
  
bool timerExpired(SoftTimer *t) {  
    if (!t->active) return false;  
    if (getTicks() - t->start >= t->duration) {  
        t->active = false;  
        return true;  
    }
```

```

    }
    return false;
}

```

Oefening

Maak drie software timers. Eén laat een LED knipperen, één triggert een buzzer na 2 seconden, één stuurt om de seconde een statusbericht. Alles moet zonder delay() werken.

5. Events en event queue

Events vormen de lijm tussen scheduler, interrupts, input, communicatie en FSM.

Een event betekent: er is iets gebeurd dat later verwerkt moet worden.

Waarom events?

Zonder events ontstaat vaak een programma vol globale vlaggen: buttonPressed, motorDone, timeoutFlag, serialReady, errorFlag.

Dat werkt tijdelijk, maar wordt snel onoverzichtelijk. Een event queue maakt de stroom expliciet.

Zonder event queue	Met event queue
Veel losse globale flags	Eén centrale stroom van gebeurtenissen
Volgorde vaak onduidelijk	Events worden in volgorde verwerkt
Modules kennen elkaar te goed	Producer en consumer zijn losser gekoppeld
Moeilijk te debuggen	Queue kan via Serial geprint worden

Eventtypes

```

typedef enum {
    EV_NONE,
    EV_BUTTON_START,
    EV_BUTTON_CANCEL,
    EV_DOOR_OPEN,
    EV_DOOR_CLOSED,
    EV_WATER_LEVEL_OK,
    EV_TIMEOUT,
    EV_MOTOR_ERROR
} EventType;

```

```

typedef struct {
    EventType type;
    int value;
} Event;

```

Ringbuffer als event queue

```

#define EVENT_QUEUE_SIZE 8

Event queue[EVENT_QUEUE_SIZE];
byte head = 0;
byte tail = 0;

```

```

bool postEvent(Event e) {
    byte next = (head + 1) % EVENT_QUEUE_SIZE;
    if (next == tail) return false; // queue full
    queue[head] = e;
    head = next;
    return true;
}

bool getEvent(Event *e) {
    if (head == tail) return false; // empty
    *e = queue[tail];
    tail = (tail + 1) % EVENT_QUEUE_SIZE;
    return true;
}

```

Oefeningen

5. Button task post EV_BUTTON_START bij een flank.
6. Serial task post events op basis van commando's zoals s, c, d.
7. Event monitor print alle events die uit de queue komen.
8. Test wat gebeurt als de queue te klein is en events sneller binnenkomen dan ze verwerkt worden.

6. Finite State Machines: basis

Een finite state machine beschrijft gedrag als een beperkt aantal toestanden en overgangen. Het systeem doet niet “alles altijd”, maar reageert afhankelijk van de huidige toestand.

Theoretische onderbouw

Een state is een betekenisvolle toestand waarin het systeem zich bevindt. Een event is iets dat een overgang kan veroorzaken. Een transition is de overgang van één state naar een andere. Een action is code die uitgevoerd wordt bij binnenkomen, verlaten of tijdens een toestand.

Term	Voorbeeld wasmachine
State	IDLE, FILLING, WASHING, RINSING, SPINNING, DONE, ERROR
Event	START gedrukt, waterniveau bereikt, timeout, deur open
Transition	FILLING -> WASHING wanneer waterniveau OK
Action	Motor starten, klep openen, buzzer activeren

Eenvoudige switch-case FSM

```
typedef enum {
    STATE_IDLE,
    STATE_RUNNING,
    STATE_DONE,
    STATE_ERROR
} State;

State state = STATE_IDLE;

void handleEvent(Event e) {
    switch (state) {
        case STATE_IDLE:
            if (e.type == EV_BUTTON_START) {
                state = STATE_RUNNING;
                Serial.println("Start");
            }
            break;

        case STATE_RUNNING:
            if (e.type == EV_TIMEOUT) {
                state = STATE_DONE;
                Serial.println("Klaar");
            }
            break;

        case STATE_DONE:
            if (e.type == EV_BUTTON_START) {
                state = STATE_IDLE;
            }
            break;

        case STATE_ERROR:
```

```

        break;
    }
}

```

Entry, do en exit

Voor grotere systemen is het nuttig om onderscheid te maken tussen acties bij binnenkomen van een state, periodieke acties terwijl de state actief is en acties bij verlaten van een state.

- Entry: motor starten, timer zetten, status printen.
- Do: sensor blijven controleren, PWM bijwerken.
- Exit: motor stoppen, klep sluiten, teller resetten.

Oefening

Maak een deur-FSM met states CLOSED, OPENING, OPEN, CLOSING en events BUTTON, LIMIT_OPEN, LIMIT_CLOSED, TIMEOUT. Gebruik LEDs om de toestand weer te geven.

7. Event-driven FSM

De FSM wordt krachtiger wanneer ze niet rechtstreeks knoppen of sensoren leest, maar events ontvangt. De scheduler en input-taken produceren events; de FSM verwerkt ze.

Architectuur

```

void loop() {
    schedulerRun();      // scant inputs, serial, timers
    processEvents();     // stuurt events naar FSM
    machineDoActions();  // periodieke acties van huidige state
}

```

Eventverwerking

```

void processEvents(void) {
    Event e;
    while (getEvent(&e)) {
        machineHandleEvent(e);
    }
}

```

Waarom dit beter is

- De FSM hoeft niet te weten waar het event vandaan komt.
- Een START-event kan komen van een knop, Serial of later van Bluetooth.
- Debuggen wordt eenvoudiger: print de eventstroom.
- Tests worden eenvoudiger: voed de FSM met kunstmatige events.

Oefening

Maak een verkeerslicht-FSM die alleen events verwerkt.

Een timer task post EV_TIMEOUT.

Een knop task post EV_PEDESTRIAN_REQUEST.

De FSM beslist wanneer de voetganger groen krijgt.

8. Meerdere FSM-modules

In een groter embedded systeem is één gigantische FSM vaak niet ideaal. Beter is een combinatie van kleinere FSMs met elk één verantwoordelijkheid.

Voorbeeldindeling

Module	Verantwoordelijkheid	Voorbeeldstates
UI FSM	Start, cancel, statusweergave	UI_IDLE, UI_ACTIVE, UI_ALARM
Process FSM	Volgorde van programma	IDLE, FILLING, WASHING, RINSING, SPINNING
Motor FSM	Motor veilig aansturen	MOTOR_OFF, MOTOR_STARTING, MOTOR_RUNNING, MOTOR_FAULT
Alarm FSM	Fouten en buzzerpatronen	ALARM_OFF, ALARM_BEEPING, ALARM_LATCHED

Communicatie tussen FSMs

FSMs kunnen events naar elkaar sturen via dezelfde queue. Bijvoorbeeld: de Process FSM vraagt MOTOR_START; de Motor FSM post MOTOR_OK of MOTOR_ERROR. Zo blijft motorlogica gescheiden van proceslogica.

Oefening

Splits een eenvoudige motoropdracht in twee modules: een UI FSM die START/STOP verwerkt, en een Motor FSM die de motorstatus beheert. De UI mag de motor niet rechtstreeks aansturen, maar alleen events posten.

9. Integratiearchitectuur

Nu komen de bouwstenen samen. De firmware bestaat uit lagen. Onderaan zitten hardware drivers. Daarboven tasks die input scannen en tijd bewaken. Events verbinden de modules. FSMs bepalen het gedrag.

Aanbevolen lagen

Laag	Voorbeelden	Regel
Hardware drivers	led_set(), motor_setPwm(), readDoor()	Geen proceslogica.
Tasks	buttonTask(), timerTask(), serialTask()	Produceren events of werken periodiek bij.
Event queue	postEvent(), getEvent()	Centrale gebeurtenisstroom.
FSMs	machineHandleEvent()	Beslissen over toestanden en acties.
Applicatie	Wasmachineprogramma	Combinatie van modules.

Globale regels voor nette firmware

- Geen delay() in tasks of FSMs.
- ISR zo kort mogelijk houden.
- Drivers mogen niet weten in welke state het systeem zit.
- FSM mag liefst geen pinnen rechtstreeks manipuleren, maar driverfuncties gebruiken.
- Events krijgen duidelijke namen.
- Elke state heeft een duidelijke betekenis.
- Fouten zijn states, geen losse vergeten flags.

Debugstrategie

- Print elke state transition.
- Print queue overflow.
- Gebruik één heartbeat LED om te tonen dat de scheduler leeft.
- Gebruik Serial-commando's om events kunstmatig te posten.

- Test elke FSM eerst zonder hardware door events te simuleren.

10. Grote oefening: Wasmachine-controller

De eindopdracht is een gesimuleerde wasmachine in Tinkercad. De opdracht combineert scheduler, software timers, event queue, FSM, actuatorsturing, sensoren, fouten en debug-output.

Functionele beschrijving

De wasmachine heeft een startknop, een deurcontact, een waterniveausensor, een motor, een waterklep, een pomp, status-LEDs en een buzzer. De machine doorloopt een eenvoudig wasprogramma: vullen, wassen, pompen, spoelen, pompen, zwieren, klaar.

Hardwarevoorstel

Functie	Tinkercad-component	Arduino-pin voorbeeld
Startknop	Pushbutton naar GND met INPUT_PULLUP	D2
Cancel/reset	Pushbutton naar GND met INPUT_PULLUP	D3
Deurcontact	Schakelaar of pushbutton	D4
Waterniveau	Potentiometer	A0
Waterklep	LED	D8
Pomp	LED	D9
Motor	DC-motor via driver of LED/PWM	D10
Buzzer	Piezo	D11
Status LEDs	Meerdere LEDs of RGB LED	D12, D13

States

```
typedef enum {
    WM_IDLE,
    WM_LOCK_CHECK,
    WM_FILLING,
    WM_WASHING,
    WM_DRAINING_AFTER_WASH,
    WM_RINSING,
    WM_DRAINING_AFTER_RINSE,
    WM_SPINNING,
    WM_DONE,
    WM_ERROR
} WashingState;
```

Events

```
typedef enum {
    EV_NONE,
    EV_START,
    EV_CANCEL,
    EV_DOOR_OPEN,
    EV_DOOR_CLOSED,
    EV_WATER_LEVEL_OK,
    EV_WATER_TIMEOUT,
    EV_PHASE_TIMEOUT,
    EV_MOTOR_FAULT,
    EV_RESET
}
```

```
} EventType;
```

State transition table

Huidige state	Event/conditie	Nieuwe state	Actie
WM_IDLE	EV_START + deur gesloten	WM_FILLING	Klep open, vultimer starten
WM_IDLE	EV_START + deur open	WM_ERROR	Buzzer, foutmelding
WM_FILLING	EV_WATER_LEVEL_OK	WM_WASHING	Klep dicht, motor traag, wastimer starten
WM_FILLING	EV_WATER_TIMEOUT	WM_ERROR	Klep dicht, fout water
WM_WASHING	EV_PHASE_TIMEOUT	WM_DRAINING_AFTER_WASH	Motor stop, pomp aan, timer starten
WM_DRAINING_AFTER_WASH	EV_PHASE_TIMEOUT	WM_RINSING	Pomp uit, klep open, timer starten
WM_RINSING	EV_PHASE_TIMEOUT	WM_DRAINING_AFTER_RINSE	Motor stop, pomp aan
WM_DRAINING_AFTER_RINSE	EV_PHASE_TIMEOUT	WM_SPINNING	Pomp uit, motor snel
WM_SPINNING	EV_PHASE_TIMEOUT	WM_DONE	Alles uit, buzzer kort
Elke actieve state	EV_DOOR_OPEN	WM_ERROR	Alles veilig uit
Elke state	EV_CANCEL	WM_IDLE	Alles uit

Taken in de scheduler

Task	Periode	Verantwoordelijkheid
buttonTask	10 ms	Debounce start en cancel; post EV_START/EV_CANCEL
doorTask	20 ms	Detecteer deur open/dicht
waterTask	50 ms	Lees A0; post EV_WATER_LEVEL_OK indien drempel bereikt
softwareTimerTask	10 ms	Controleer fase- en fouttimers
machineTask	20 ms	Periodieke do-acties van huidige state
serialTask	50 ms	Debugcommando's lezen
statusTask	100 ms	LEDs en heartbeat bijwerken

Skeletoncode

```
void loop() {
    schedulerRun();
    processEvents();
    washingMachineDo();
}

void washingMachineHandleEvent(Event e) {
    switch (wmState) {
        case WM_IDLE:
            if (e.type == EV_START) {
                if (doorClosed()) {
                    enterState(WM_FILLING);
                } else {
                    enterError("Deur open bij start");
                }
            }
            break;
    }
}
```

```

case WM_FILLING:
    if (e.type == EV_WATER_LEVEL_OK) {
        enterState(WM_WASHING);
    } else if (e.type == EV_WATER_TIMEOUT) {
        enterError("Geen waterniveau bereikt");
    } else if (e.type == EV_DOOR_OPEN) {
        enterError("Deur geopend tijdens vullen");
    }
    break;

case WM_WASHING:
    if (e.type == EV_PHASE_TIMEOUT) {
        enterState(WM_DRAINING_AFTER_WASH);
    } else if (e.type == EV_DOOR_OPEN) {
        enterError("Deur geopend tijdens wassen");
    }
    break;

default:
    break;
}
}

```

Entry-acties

```

void enterState(WashingState newState) {
    exitState(wmState);
    wmState = newState;

    Serial.print("Nieuwe state: ");
    Serial.println(stateName(wmState));

    switch (wmState) {
        case WM_IDLE:
            allActuatorsOff();
            break;

        case WM_FILLING:
            valveOn();
            pumpOff();
            motorOff();
            timerStart(&waterTimeoutTimer, 8000);
            break;

        case WM_WASHING:
            valveOff();
            motorSlow();
            timerStart(&phaseTimer, 10000);

```

```

        break;

    case WM_SPINNING:
        pumpOff();
        motorFast();
        timerStart(&phaseTimer, 7000);
        break;

    case WM_DONE:
        allActuatorsOff();
        tone(BUZZER_PIN, 1200, 300);
        break;

    case WM_ERROR:
        allActuatorsOff();
        tone(BUZZER_PIN, 300);
        break;
    }
}

```

Minimumvereisten

9. Geen delay() gebruiken in de hoofdlogica.
10. Minstens vijf duidelijke states implementeren.
11. Minstens één software timer gebruiken.
12. Startknop debouncen.
13. Deur openen tijdens actief programma leidt tot ERROR.
14. Elke state transition wordt op Serial geprint.
15. Alle actuatorfuncties zitten in aparte driverfuncties.
16. Het programma kan na ERROR gereset worden.

Vrijblijvende uitbreidingen (vooral voorzien voor lessen in de klas)

- Motor-FSM apart maken van Process-FSM.
- Waterniveau realistisch laten stijgen via potentiometer of simulatievariabele.
- Serial command shell toevoegen: start, cancel, status, fault.
- Queue overflow detecteren en rapporteren.
- CPU-load schatten via idle counter.
- EEPROM gebruiken om foutcodes te bewaren.
- Wasmachineprogramma's toevoegen: kort, normaal, intensief.
- Table-driven FSM proberen voor enkele states.

Samenvatting

Deze bundel introduceert het concept waarbij je kan evolueren van Arduino-code die sequentieel en blokkerend werkt naar firmware die modulair, responsief en event-driven is.

De cooperative scheduler beheert tijd en periodieke taken.

De event queue koppelt detectie los van verwerking.

De FSM maakt gedrag expliciet en controleerbaar.

De wasmachine-oefening is geschikt als integratieproject omdat ze timing, veiligheid, foutafhandeling, actuatoren en gebruikersinteractie combineert in één begrijpelijk systeem.

Evaluatiecriteria

Criterium	Basis	Goed	Sterk
Timing	Werkt grotendeels zonder delay	Alle timing non-blocking	Jitter en taskduur worden besproken
Scheduler	Taken zijn gescheiden	Taakstructuur met perioden	Uitbreidbaar en debugbaar schedulerontwerp
Events	Enkele events gebruikt	Event queue correct toegepast	Overflow en volgorde worden gecontroleerd
FSM	Duidelijke states	Transitions zijn logisch en veilig	Entry/exit/do-acties netjes gescheiden
Modulariteit	Code is leesbaar	Drivers, tasks en FSM gescheiden	Meerdere FSMs of lagenarchitectuur
Foutafhandeling	Basis ERROR-state	Veilige actuatorstop bij fouten	Reset, foutcodes en testscenario's
Debugging	Serial output aanwezig	Transitions en events traceerbaar	Gerichte testcommando's en diagnose